ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

ABP · INGENIERÍA INFORMÁTICA · UNIVERSIDAD DE VIGO

Sesión 4

Desarrollo de software en la era de la IA (I)

De la programación tradicional al desarrollo basado en especificaciones

4º curso · 30 de septiembre de 2026

Prof. Gerardo Rodríguez Martínez

Cuando la IA escribe el código, la especificación pasa a ser la fuente de verdad

Objetivos de hoy

Al terminar esta sesión deberíais poder responder a tres preguntas

1

¿Cómo ha cambiado el trabajo del programador?

Del autocompletado al chat y de ahí a los agentes autónomos: qué cambia y qué sigue igual.

2

¿Qué es el «vibe coding» y qué riesgos tiene?

Programar a base de prompts sueltos: cuándo sirve y dónde se rompe.

3

¿Qué es el desarrollo basado en especificaciones?

Cómo se diseña una especificación clara y verificable, con un ejemplo completo de principio a fin.

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Del autocompletado a los agentes

En cinco años, la IA ha pasado de sugerir la siguiente línea a completar tareas enteras. Cada etapa cambia qué parte del trabajo hace la máquina y qué parte sigue siendo nuestra.

2021 · Autocompletado

Sugiere la línea o la función siguiente mientras escribes (p. ej. GitHub Copilot). Tú sigues escribiendo el código.

2022-23 · Chat

Pides código en lenguaje natural y lo copias (p. ej. ChatGPT). Aparece la tentación de pegar sin leer.

2024 · IA dentro del editor

El asistente ve tu proyecto y edita varios ficheros a la vez desde el IDE.

2025-26 · Agentes autónomos

Planifican, ejecutan comandos, lanzan tests y abren pull requests por su cuenta (p. ej. Claude Code, Codex, Copilot agent).

El nuevo papel del programador

La IA no elimina el trabajo de programar: lo desplaza. Menos tiempo tecleando código y más tiempo decidiendo qué construir, revisando y verificando.

Cuanto más autónoma es la herramienta, más importa saber explicarle qué queremos y comprobar que lo ha hecho bien.

Lo que cambia

- Escribir código a mano pasa a ser una parte menor del trabajo
- Crece el peso de especificar, revisar y probar
- Se puede prototipar en minutos lo que antes llevaba días

Lo que no cambia

- **Entender el problema y el dominio sigue siendo cosa vuestra**
- **El código que entregáis es vuestra responsabilidad, lo escriba quien lo escriba**
- **Sin pruebas, no sabéis si funciona — tampoco si lo escribió una IA**

Tres formas de programar hoy

Ninguna es «la correcta» para todo: cambian la fuente de verdad y el tipo de riesgo

TR

Tradicional

La persona escribe el código (con o sin autocompletado). El código es la única fuente de verdad. Control total, pero lento.

VC

Vibe coding

Prompts sueltos, se acepta lo que genera la IA y se itera «a ojo» hasta que parece funcionar. Rapidísimo para empezar; frágil para mantener.

SDD

Por especificación

Se escribe primero una especificación verificable (requisitos, criterios de aceptación, casos límite) y la IA la traduce en código.

Vibe coding: qué es y cuándo tiene sentido

Término acuñado por Andrej Karpathy en febrero de 2025: programar «dejándose llevar» por la IA, aceptando el código sin leerlo y pidiendo cambios hasta que funcione.

No es un insulto ni algo prohibido: es una herramienta con un rango de uso concreto. El problema aparece cuando el prototipo se convierte, sin querer, en el producto.

Funciona bien para...

- Prototipos desechables y pruebas de concepto
- Explorar una librería o un dataset nuevo
- Scripts de un solo uso que nadie va a mantener

Se rompe cuando...

- **El código crece y nadie sabe por qué está hecho así**
- **Otra persona del equipo tiene que modificarlo**
- **Hay que demostrar que cumple unos requisitos (como en vuestros sprints)**

Percepción frente a realidad

Lo que dicen algunos estudios recientes sobre programar con IA

84 %

de los desarrolladores usan o planean usar herramientas de IA

Stack Overflow Developer Survey 2025

46 %

desconfía de la exactitud de sus respuestas (frente a un 33 % que confía)

Stack Overflow Developer Survey 2025

66 %

señala como principal frustración las soluciones de IA «casi correctas, pero no del todo»

Stack Overflow Developer Survey 2025

Percepción ≠ realidad: en 2025, METR midió a desarrolladores expertos un 19 % más lentos con IA mientras creían ir un 20 % más rápido. En 2026 estima que las herramientas actuales ya les aceleran, pero reconoce que no puede medirlo con fiabilidad. Sin verificación objetiva, es fácil engañarse.

Los tres riesgos de programar sin especificación

Aparecen con cualquier asistente, pero se multiplican con el vibe coding

!

Alucinaciones

Funciones o librerías que no existen, parámetros inventados, código «casi correcto» que falla en los casos que nadie probó.

€

Deuda técnica oculta

Código duplicado, decisiones incoherentes entre ficheros y soluciones que nadie del equipo entiende del todo.

?

Falta de trazabilidad

No queda escrito por qué el código hace lo que hace ni qué requisito cubre: imposible revisar o defender el diseño.

Desarrollo basado en especificaciones

Primero se escribe qué debe hacer el sistema, de forma clara y verificable; después la IA genera el código a partir de esa especificación.

La especificación pasa a ser la fuente de verdad: se revisa, se versiona y se discute. El código se puede regenerar; la especificación, no.

Una buena especificación incluye

- Objetivo o historia de usuario, y entradas / salidas
- Requisitos funcionales numerados
- Criterios de aceptación comprobables (Dado / Cuando / Entonces)
- Casos límite, restricciones y lo que queda fuera

Por qué os importa hoy

- **Vuestras historias de usuario y casos de uso del Sprint 1 ya son el embrión de una especificación**
- **Cada criterio de aceptación puede convertirse en un test automático**

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

El flujo de trabajo

La persona decide y revisa en cada paso; la IA acelera la ejecución. En la sesión 5 lo aplicaréis de principio a fin.

1 **Especificación**
Qué debe hacer y cómo se comprueba

›

2 **Plan técnico**
Arquitectura, datos, decisiones

›

3 **Tareas**
Pasos pequeños y revisables

›

4 **Código**
Generado por la IA, tarea a tarea

›

5 **Verificación**
Tests y revisión humana

Herramientas que siguen este flujo: GitHub Spec Kit (especificar → planificar → tareas → implementar → converger) o Kiro (requirements → design → tasks). Pero el método funciona con cualquier asistente.

Comparativa de los tres enfoques

Elegid según el tipo de tarea, no por costumbre

	Tradicional	Vibe coding	Basado en especificaciones
Fuente de verdad	El código	Ninguna (el chat)	**La especificación**
Velocidad inicial	Baja	Muy alta	**Media**
Mantenibilidad	Alta si hay disciplina	Baja	**Alta**
Trazabilidad	Media (commits, comentarios)	Casi nula	**Alta: requisito → test → código**
Riesgo principal	Lentitud	Deuda técnica oculta, alucinaciones	**Especificaciones ambiguas o incompletas**
Encaja en…	Piezas críticas o muy específicas	Prototipos y exploración	**Funcionalidades del producto**

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Un ejemplo completo, paso a paso

Lo hacemos juntos en clase; después lo repetiréis por libre con otra funcionalidad

```
# Contexto: esto ya existe en el proyecto
# ranking_similares(item_id) -> [(id, score), ...]
#   ranking COMPLETO, ordenado por similitud

def recomendar(item_id, n=10):
    return [{"id": i, "titulo": ..., "score": s}
            for i, s in ranking_similares(item_id)[:n]]
```

Encargo: «Añadir la posibilidad de filtrar las recomendaciones por categoría.»

1. **El prompt suelto**
   Lo pedimos «a lo rápido» y miramos qué devuelve
2. **Diseñar la especificación**
   Seis pasos y las preguntas que destapan los casos límite
3. **Generar código y tests**
   Conversación nueva, especificación completa
4. **Verificar y comparar**
   Criterio a criterio: ¿qué versión cumple lo pedido?

Paso 1 · El prompt suelto

«Añade un filtro por categoría a la función recomendar()» — respuesta típica de un asistente:

```
def recomendar(item_id, n=10, categoria=None):
    recs = [{"id": i, "titulo": ..., "score": s}
            for i, s in ranking_similares(item_id)[:n]]
    if categoria:
        recs = [r for r in recs
                if categoria in
                catalogo[r["id"]]["categorias"]]
    return recs
```

Parece correcto y con un ejemplo sencillo funciona. Pero toma varias decisiones que nadie ha tomado de forma consciente.

Decisiones implícitas

- **Filtra después de truncar a n**
  Pides 10 comedias y devuelve 2 aunque haya 50
- **Compara el texto exacto**
  «Comedia» ≠ «comedia» ≠ «Comédia»
- **Solo una categoría**
  ¿«Terror o ciencia ficción»? Nadie lo decidió
- **Cambia una función que ya se usa**
  Puede romper otras partes del código
- **Sin tests ni casos límite**
  ¿Y si la categoría no existe? ¿Y si n = 0?

Paso 2 · Cómo se diseña una buena especificación

Seis pasos, cada uno con la pregunta que hay que responder

1 Objetivo

¿Quién lo usa y para qué?

Historia de usuario: como usuario, quiero filtrar por una o varias categorías…

2 Entradas y salidas

¿Qué recibe y qué devuelve exactamente?

recomendar_por_categoria(item_id, n, categorias) → lista de hasta n dicts

3 Requisitos

¿Qué debe hacer, sin decir cómo?

RF1: filtrar sobre el ranking completo y truncar a n después

4 Criterios de aceptación

¿Cómo sabré que funciona?

Dado…, cuando…, entonces… con datos concretos

5 Casos límite

¿Y si…?

Categoría inexistente, menos de n resultados, ítem inexistente, n ≤ 0

6 Restricciones y fuera de alcance

¿Qué no se puede tocar y qué no se hace?

No modificar recomendar(); sin dependencias nuevas; sin UI

Criterios de aceptación: de vago a comprobable

La regla: con datos concretos, cualquiera —persona o IA— debe poder decir «cumple» o «no cumple»

Vago	Comprobable (Dado / Cuando / Entonces)
Que filtre bien por categoría.	Dado un ranking donde el 2º ítem de ciencia ficción está en 4ª posición, cuando pido n=2 con «Ciencia ficción», entonces recibo esos 2 ítems, en orden.
Que no le afecten las mayúsculas.	Dado un ítem con categoría «ciencia ficcion», cuando pido « CIENCIA FICCION», entonces el ítem aparece.
Que se puedan elegir varias.	Cuando pido [«Terror», «Comedia»], entonces recibo los ítems de cualquiera de las dos, en el orden del ranking.
Que no falle.	Dado que no hay ningún «Western», cuando lo pido, entonces recibo una lista vacía y no un error.

Trucos: al menos un criterio por requisito · cada «¿y si…?» es un caso límite · si no sabéis el resultado esperado, la IA tampoco lo sabrá.

La especificación del ejemplo

RF-07 · Filtro de recomendaciones por categoría (versión completa en la guía y en Sesion4_Demo)

Requisitos funcionales

RF1 Filtrar sobre el ranking completo; truncar a n después

RF2 Conservar el orden por similitud

RF3 Varias categorías: basta con tener una (O lógico)

RF4 Ignorar mayúsculas, tildes y espacios sobrantes

RF5 Sin categorías: mismo resultado que recomendar()

Casos límite: categoría inexistente → lista vacía · menos de n → los que haya · ítem inexistente → KeyError · n ≤ 0 → ValueError

	Criterio de aceptación (resumido)
CA1	n=2, «Ciencia ficción» → los 2 de ciencia ficción aunque uno esté 4º
CA2	[«Terror», «Comedia»] → ambos, en orden
CA3	« CIENCIA FICCION» encuentra «ciencia ficcion»
CA4	Sin categorías → igual que recomendar()
CA5	«Western» → lista vacía, sin error
CA6	Ítem inexistente o n ≤ 0 → error claro

Paso 3 · Código y tests generados desde la especificación

Conversación nueva + especificación completa + «genera el código y un test por criterio»

recomendador.py

```
def recomendar_por_categoria(item_id, n=10,
                             categorias=None):
    if not isinstance(n, int) or n <= 0:
        raise ValueError("n debe ser positivo")
    if item_id not in catalogo:
        raise KeyError(item_id)
    buscadas = {_normalizar(c)
                for c in (categorias or [])}   # RF4
    resultado = []
    for i, score in ranking_similares(item_id):  # RF1
        cats = {_normalizar(c) for c in ...}
        if buscadas and not (buscadas & cats):   # RF3
            continue
        resultado.append({...})
        if len(resultado) == n:                  # RF1
            break
    return resultado                             # RF2
```

test_recomendador.py (uno por CA)

```
def test_ca1_filtra_antes_de_truncar():
    # D es 4º en el ranking,
    # pero debe aparecer con n=2
    res = recomendar_por_categoria(
        "A", 2, ["Ciencia ficción"])
    assert ids(res) == ["B", "D"]

def test_ca5_categoria_inexistente():
    res = recomendar_por_categoria(
        "A", 5, ["Western"])
    assert res == []
```

Trazabilidad: cada comentario remite al requisito que implementa y cada test lleva el nombre del criterio que comprueba.

Paso 4 · Verificación con los mismos 6 criterios

Resultado real al ejecutar los criterios de aceptación contra cada versión

2 / 6

Prompt suelto

Solo cumple «sin filtro» (CA4) y «categoría inexistente» (CA5). Falla lo importante: CA1, CA2, CA3 y CA6.

6 / 6

Desde la especificación

Cumple todos los criterios. Si alguno fallara, sabríamos exactamente cuál y por qué.

Coste de cada versión

- Prompt suelto: 1 minuto… y los errores quedan escondidos
- Especificación: ~15 minutos pensando el problema
- Ese tiempo no se «pierde»: hay que pensarlo igualmente, antes o cuando falle en producción

Mensaje: la IA no «se equivocó» en la versión 1: hizo exactamente lo que se le pidió. El problema era la petición. La especificación convierte lo que teníamos en la cabeza en algo que se puede comprobar.

Checklist: ¿es buena vuestra especificación?

Pasadla antes de pegar la especificación en un asistente de IA

- [ ] Dice qué debe hacer el sistema, no cómo programarlo
- [ ] Los criterios usan datos concretos y resultado esperado
- [ ] Entradas y salidas definidas: nombres, tipos, formato del resultado
- [ ] Hay casos límite: vacío, inexistente, fuera de rango, empates
- [ ] Requisitos numerados, uno por idea
- [ ] Dice qué no se puede tocar y qué queda fuera de alcance
- [ ] Cada requisito tiene al menos un criterio de aceptación
- [ ] Otra persona del equipo la entiende sin preguntaros nada

La prueba de fuego: ¿podríais escribir un test a partir de cada requisito?

ESCOLA SUPERIOR DE ENXEÑARÍA INFORMÁTICA

Ahora os toca: ejercicio por libre

En equipo, antes de la sesión 5 (07/10) · Sesion4_Ficha_Alumnado.docx

Repetid el ejemplo de hoy con otra funcionalidad. Tiempo orientativo: 45-60 minutos.

Elegid la funcionalidad

Opción A (recomendada): una historia «Must» de vuestro backlog. Opción B: filtrar recomendaciones por un rango numérico (año, precio, duración…).

Ronda 1 y ronda 2

Primero con prompts sueltos; después diseñad la especificación con los seis pasos y generad código y tests en una conversación nueva.

Comparad y anotadlo

Rellenad la tabla de comparación y la bitácora de la ficha. La sesión 5 empieza con una puesta en común de lo que os haya pasado.

No se evalúa, pero es el mejor entrenamiento para el taller de la sesión 5 y para vuestro Sprint 2.

Antes de la próxima sesión

Práctica · 28-30/09

Acompañamiento del Sprint 1 (entrega 09/10): vuestras historias de usuario y casos de uso son la base de una buena especificación.

Hasta el 07/10

Ejercicio por libre de la ficha: una funcionalidad, dos rondas, comparación y bitácora.

Teoría · 07/10

Sesión 5: desarrollo con IA (II) — flujo completo especificación → plan → tareas → código → verificación, agentes y buenas prácticas.

¿Preguntas?